



BITS Pilani

Pilani Campus

Deep Neural Network

Deep Neural Network



Disclaimer and Acknowledgement



- The content for these slides has been obtained from books and various other source on the Internet
- I here by acknowledge all the contributors for their material and inputs.
- I have provided source information wherever necessary
- I have added and modified the content to suit the requirements of the course

Session Agenda



- Training Error and Generalization Error
- Fit of the model
- Model complexity
- Regularization
 - l_2 regularization
 - Dataset augmentation
 - Early stopping
 - Ensemble methods
 - Dropout



Training Error and Generalization Error

- **Training error** is the error of our model as calculated on the training dataset.
 - Obtained while training the model.
 - **Generalization error** is the expectation of our model's error, if an infinite stream of additional data examples drawn from the same underlying data distribution as the original sample were applied on the model.
 - Cannot be computed, but estimated.
 - Estimate the generalization error by applying the model to an independent test set,
 - constituted of a random selection of data examples that were withheld from the training set
-

Factors that influence the generalizability of a model

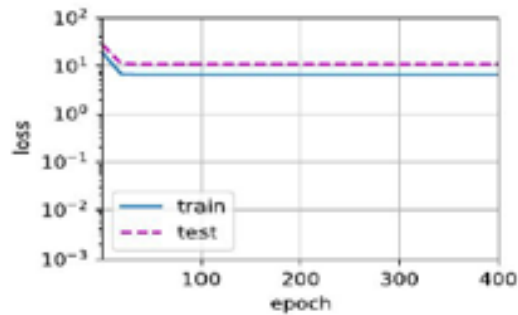
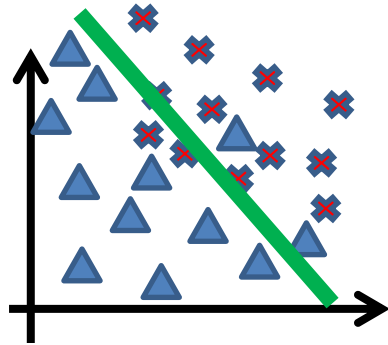


1. The number of tunable parameters.
 - When the number of tunable parameters, called the degrees of freedom, is large, models tend to be more susceptible to overfitting.
 2. The values taken by the parameters.
 - When weights can take a wider range of values, models can be more susceptible to overfitting.
 3. The number of training examples.
 - It is trivially easy to overfit a dataset containing only one or two examples even if your model is simple.
 - But overfitting a dataset with millions of examples requires an extremely flexible model.
-

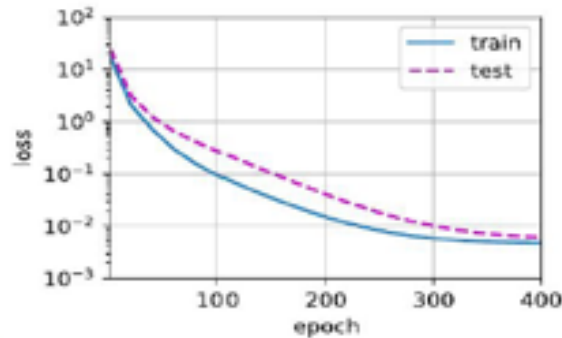
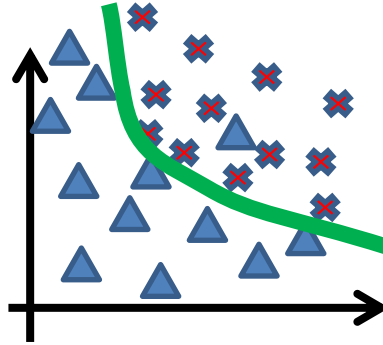
Fit of the model



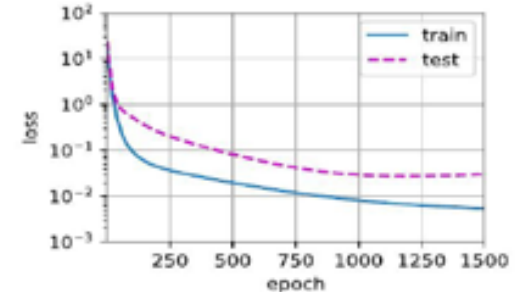
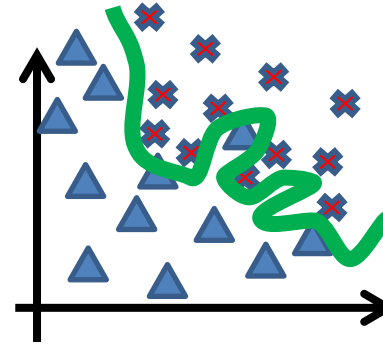
Underfitting



High Training loss
High Validation loss
Little gap between both

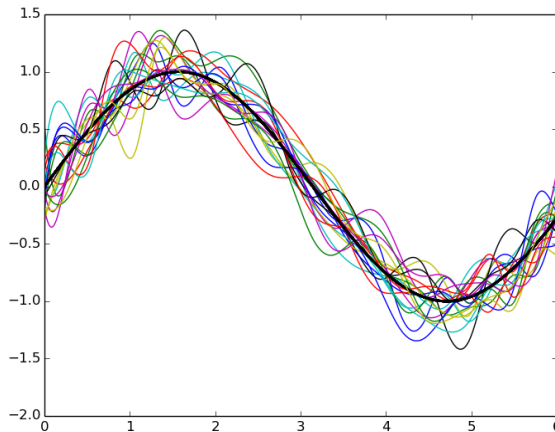
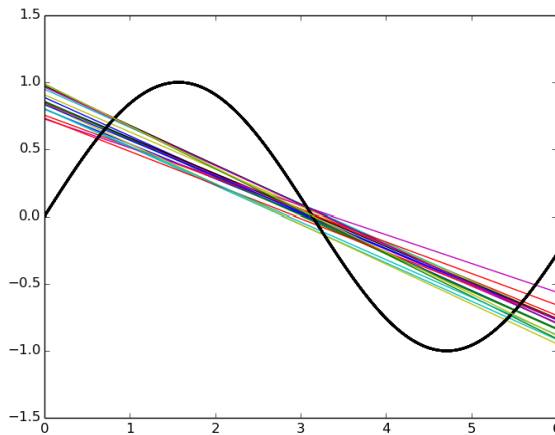


Low Training loss
Low Validation loss



Low Training loss
High Validation loss

Bias- variance



- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)
- On the other hand, complex models trained on different samples of the data are very different from each other (high variance)

Simple model: high bias, low variance

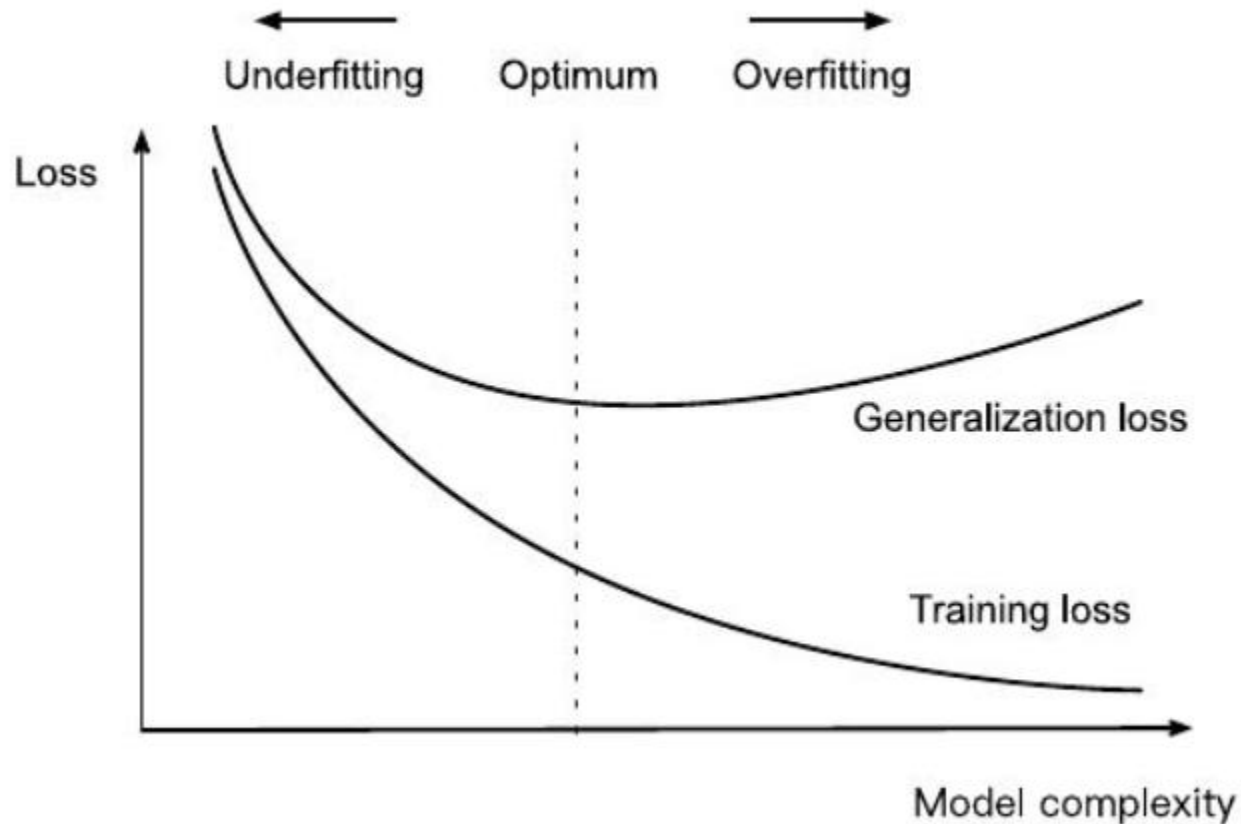
Complex model: low bias, high variance

Model complexity



- Simple models and abundant data
 - Expect the generalization error to resemble the training error.
- More complex models and fewer examples
 - Expect the training error to go down but the generalization gap to grow.
- Model complexity
 - A model with more parameters might be considered more complex.
 - A model whose parameters can take a wider range of values might be more complex.
 - A neural network model that takes more training iterations are more complex, and
 - One subject to early stopping (fewer training iterations) are less complex.

Model complexity



Model complexity



- Let there be n training points and m test (validation) points

$$\begin{aligned} \text{train}_{err} &= \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2 \\ \text{test}_{err} &= \frac{1}{m} \sum_{i=n+1}^{n+m} (y_i - \hat{f}(x_i))^2 \end{aligned}$$

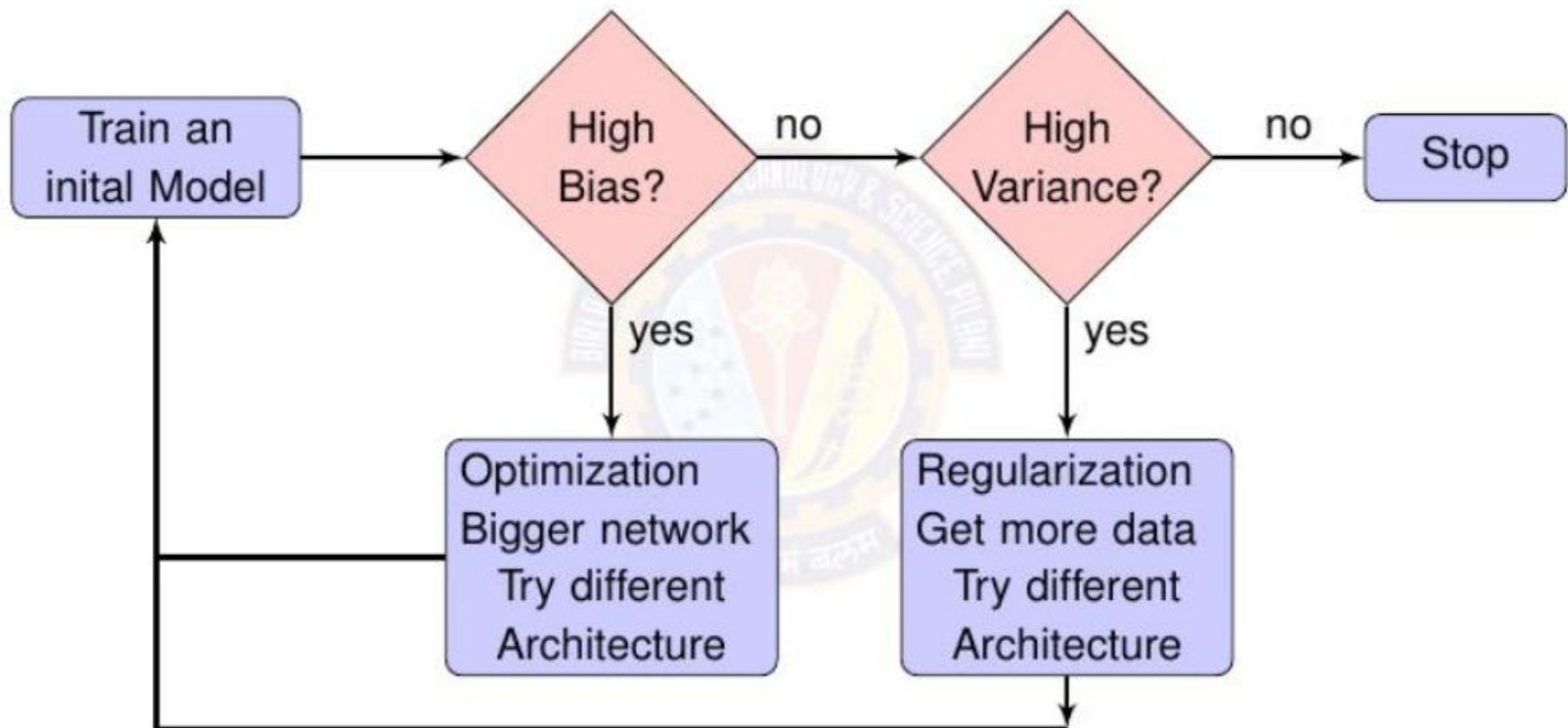
- As the model complexity increases train_{err} becomes overly optimistic and gives us a wrong picture of how close $\hat{f}$ is to f
- The validation error gives the real picture of how close $\hat{f}$ is to f

Model selection



- Model selection is the process of selecting the final model after evaluating several candidate models.
- With MLPs, compare models with
 - different numbers of hidden layers,
 - different numbers of hidden units
 - different activation functions applied to each hidden layer.
- We should touch the test data once, to assess the very best model or to compare a small number of models to each other
- Use Validation dataset to determine the best among our candidate models
- In deep learning, with millions of data available, the split is generally
 - Training = 98-99 % of the original dataset
 - Validation = 1-2 % of training dataset
 - Testing = 1-2 % of the original dataset

Model selection



Model complexity



- Why do we care about this bias variance tradeoff and model complexity?
- Deep Neural networks are highly complex models. Many parameters, many nonlinearities.
- It is easy for them to overfit and drive training error to 0.
- Hence we need some form of regularization.

Different forms of regularization



- l_2 regularization
- Dataset augmentation
- Early stopping
- Ensemble methods
- Dropout

l_2 regularization- weight decay



Regularized Cost function

$$\text{Objective function} = \mathcal{L}(w, b) + \frac{\lambda}{2} ||w||^2$$

Add the norm as a penalty term to the problem of minimizing the loss. This will ensure that the weight vector is small.

Regularized Cost function – Logistic regression

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

$$w_{t+1} = w_t - \eta \nabla L(w_t) - \eta \lambda w_t$$

w_0 is not regularized

Regularized Cost function – Neural network

$$J_{\text{regularized}} = \underbrace{-\frac{1}{m} \sum_{i=1}^m (y^{(i)} \log(a^{[L](i)}) + (1 - y^{(i)}) \log(1 - a^{[L](i)}))}_{\text{cross-entropy cost}} + \underbrace{\frac{1}{m} \frac{\lambda}{2} \sum_l \sum_k \sum_j W_{k,j}^{[l]2}}_{\text{L2 regularization cost}}$$

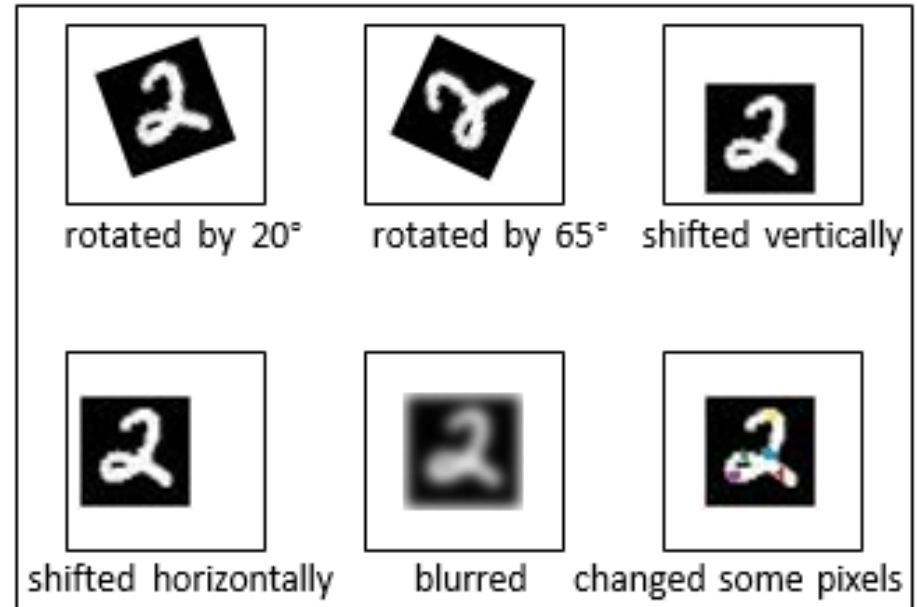
Dataset augmentation



label = 2



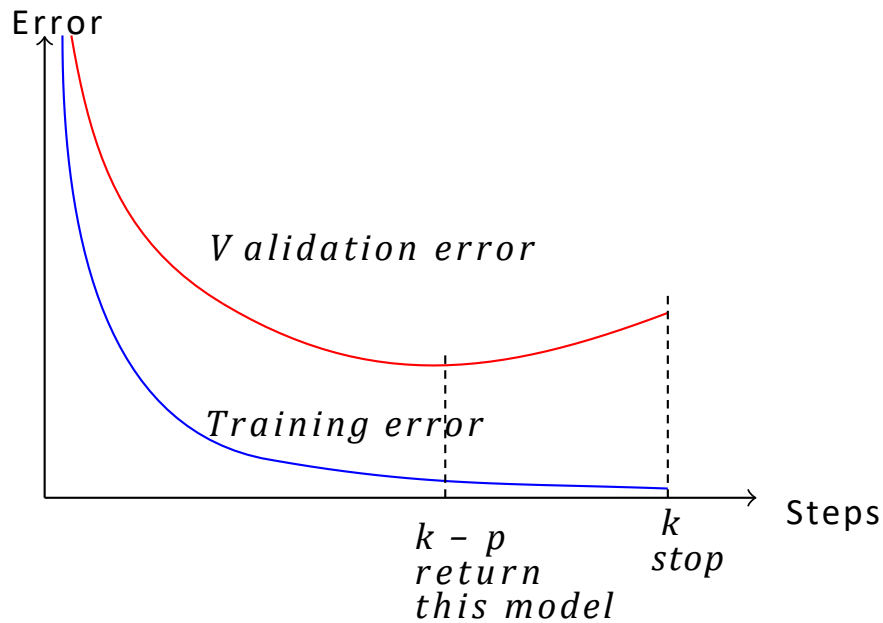
[given training data] We exploit the fact that certain transformations to the image do not change the label of the image.



- Typically, More data = better learning
- Works well for image classification / object recognition tasks Also shown to work well for speech
- For some tasks it may not be clear how to generate such data

[augmented data = created using some knowledge of the task]

Early stopping



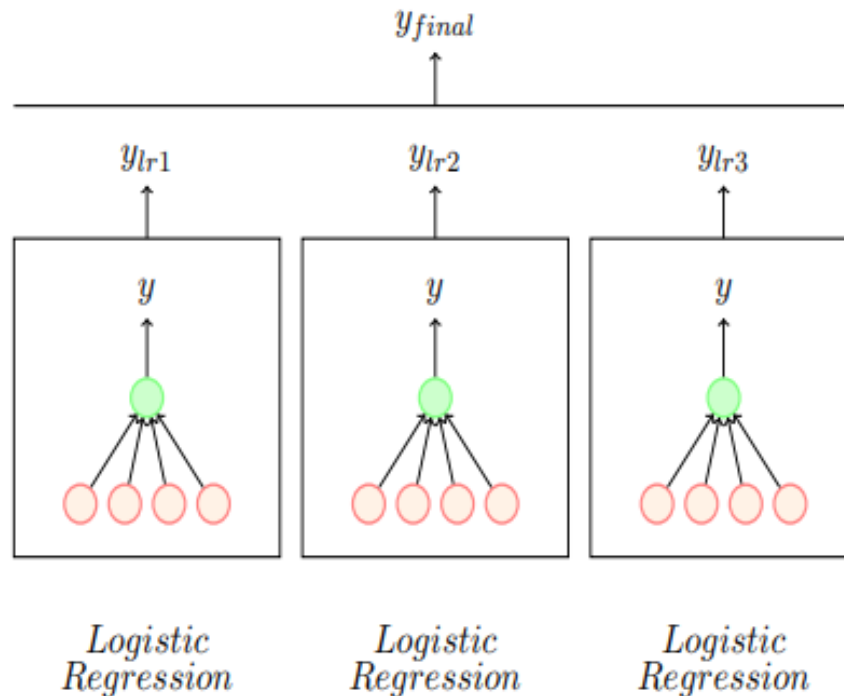
- Track the validation error
- Have a patience parameter p
- If you are at step k and there was no improvement in validation error in the previous p steps then stop training and return the model stored at step $k - p$
- Basically, stop the training early before it drives the training error to 0 and blows up the validation error

Early stopping



```
>>> callback = tf.keras.callbacks.EarlyStopping(monitor='loss', patience=3)
>>> # This callback will stop the training when there is no improvement in
>>> # the loss for three consecutive epochs.
>>> model = tf.keras.models.Sequential([tf.keras.layers.Dense(10)])
>>> model.compile(tf.keras.optimizers.SGD(), loss='mse')
>>> history = model.fit(np.arange(100).reshape(5, 20), np.zeros(5),
...                     epochs=10, batch_size=1, callbacks=[callback],
...                     verbose=0)
>>> len(history.history['loss']) # Only 4 epochs are run.
```

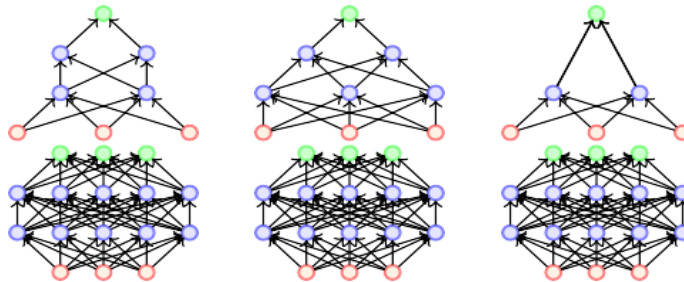
Ensemble - Bagging



Each model trained with a different sample of the data (sampling with replacement)

Ensemble - Bagging

- Typically model averaging(bagging ensemble) always helps
- Training several large neural networks for making an ensemble is prohibitively expensive
- Option 1: Train several neural networks having different architectures(Obviously expensive)
- Option 2: Train multiple instances of the same network using different training samples (again expensive)
- Even if we manage to train with option 1 or option 2, combining several models at test time is infeasible in real time applications



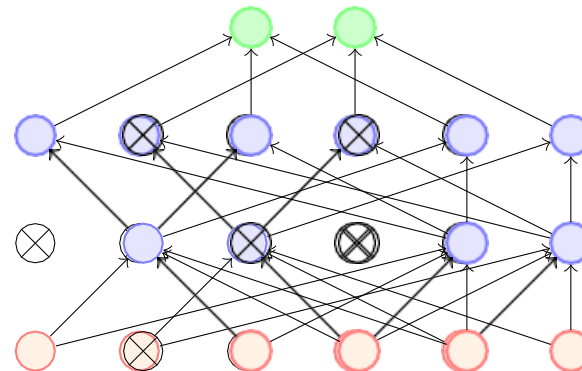
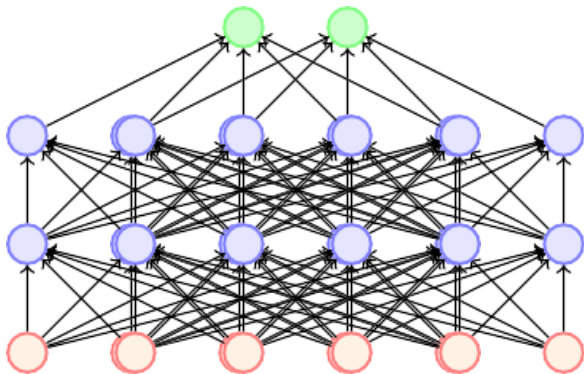
Drop out



- Dropout is a technique which addresses both these issues.
- Effectively it allows training several neural networks without any significant computational overhead.
- Also gives an efficient approximate way of combining exponentially many different neural networks.

Drop out

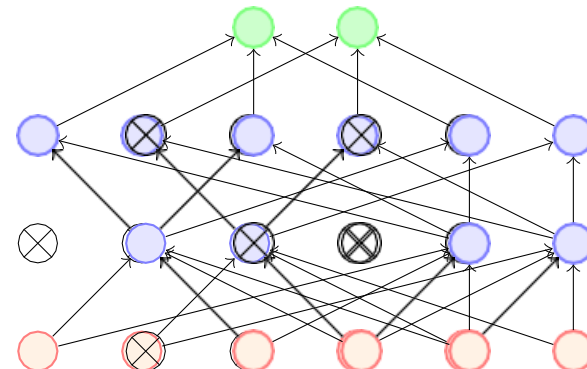
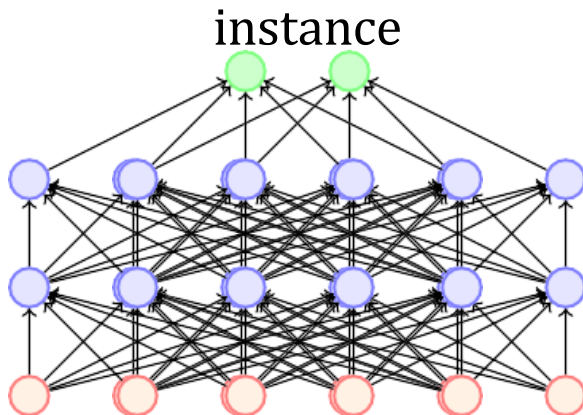
- Dropout refers to dropping out units
- Temporarily remove a node and all its incoming/outgoing connections resulting in a thinned network
- Each node is retained with a fixed probability (typically $p = 0.5$) for hidden nodes and $p = 0.8$ for visible nodes



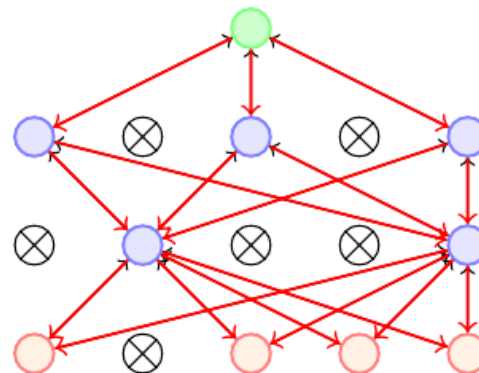
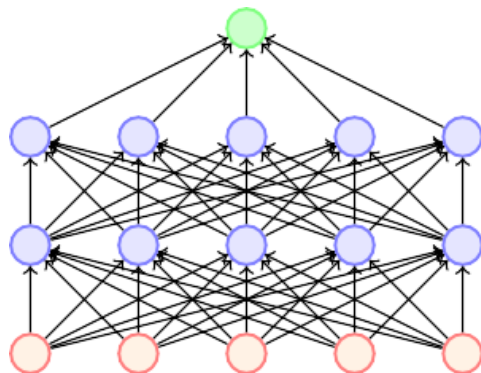
Drop out



- Suppose a neural network has n nodes
- Using the dropout idea, each node can be retained or dropped
- For example, in the above case we drop 5 nodes to get a thinned network
Given a total of n nodes, what are the total number of thinned networks that can be formed? 2^n
- we cannot possibly train so many networks
- **Trick:** (1) Share the weights across all the networks
(2) Sample a different network for each training instance

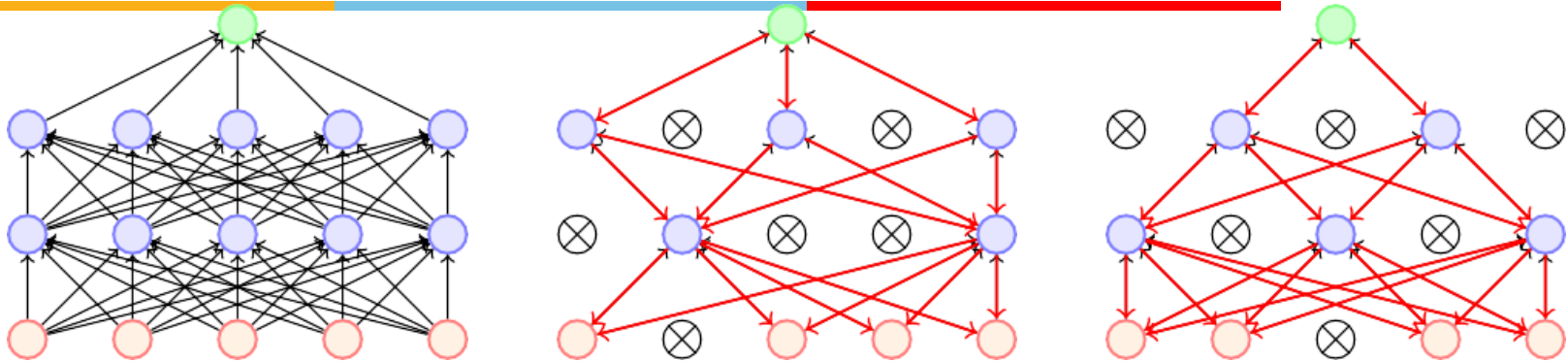


Drop out



- We initialize all the parameters (weights) of the network and start training
- For the first training instance (or mini-batch), we apply dropout resulting in the thinned network
- We compute the loss and back propagate
- **Which parameters will we update? Only those which are active**

Drop out



- For the second training instance (or mini-batch), we again apply dropout resulting in a different thinned network
- We again compute the loss and backpropagate to the active weights
- If the weight was active for both the training instances then it would have received two updates by now
- If the weight was active for only one of the training instances then it would have received only one updates by now
- Parameter sharing ensures that no model has untrained or poorly trained parameters

Drop out



- Prevents hidden units from coadoption
- Dropout gives a smaller neural network, giving the effect of regularization.
- In general,
 - Vary keep probability (0.5 to 0.8) for each hidden layer.
 - The input layer has a keep probability of 1.0 or 0.9.
 - The output layer has a keep probability of 1.0.

References



- <https://www.cs.toronto.edu/~hinton/absps/JMLRdropout.pdf>
- Ref TB Dive into Deep Learning Sections 5.4, 5.5, 5.6 online version
- IIT M CS7015 (Deep Learning) : Lecture 8



Thank You All !